

24. Swap Nodes in Pairs

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 8, 2026

Problem Summary

Given the head of a linked list, swap every two adjacent nodes and return the head. The node values are swapped, not the nodes themselves.

Approach

Traverse the list in steps of two. At each pair, swap the values of `curr` and `curr.next` using a temp variable, then jump `curr` two nodes ahead. Repeat until fewer than two nodes remain.

Algorithm

```
If head == null || head.next == null return head.  
Set curr = head.  
While curr != null && curr.next != null:  
    • Swap curr.val and curr.next.val via temp.  
    • curr = curr.next.next.  
Return head.
```

Key Insights

- Swapping **values** instead of **pointers** drastically simplifies the logic — no prev/next rewiring needed.
- Base case handles both empty list and single node — both need no swapping.
- `curr.next.next` skips exactly one pair per iteration.
- head remains valid throughout since no structural changes are made — safe to return directly.

Points to Remember

1. **Value swap vs pointer swap:** value swap is always simpler — reach for it first unless the problem forbids it.
2. `curr.next.next` is the standard two-step jump for pairwise traversal.
3. **Base case pattern:** `head == null || head.next == null` — reuse this for any problem that operates on pairs.
4. The pointer-swap version of this problem is a good exercise: you need `prev -> [A -> B] -> rest`, rewire to `prev -> [B -> A] -> rest` — practice it separately as it generalizes to reverse in groups of `k` (LC 25).

Observations

- Value-swap approach is $O(1)$ extra space and much simpler than pointer rewiring.
- Pointer-swap approach (what interviewers typically expect) requires tracking `prev`, `curr`, `curr.next`, and `curr.next.next` — significantly more complex.
- This solution would fail if the problem explicitly said "must not modify node values" — always check constraints.
- Odd-length lists are handled naturally — last solo node is never entered in the while condition.

Time Complexity & Space Complexity

$O(n)$ — single pass through all nodes, two at a time.

$O(1)$ — only a temp variable and pointer, no extra structure.

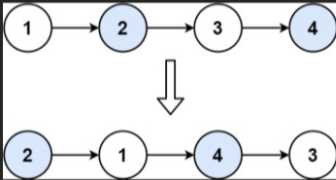
Linked List

DescriptionEditorialSolutionsSubmit

24. Swap Nodes in PairsSolved

MediumTopicsCompanies

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:
Input: head = [1,2,3,4]
Output: [2,1,4,3]
Explanation:


Example 2:
Input: head = []
Output: []

Example 3:
Input: head = [1]
Output: [1]

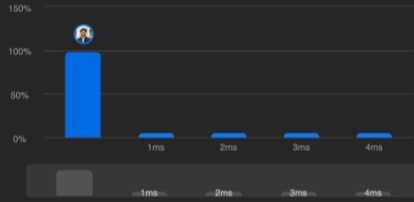
Example 4:
13.3K203101 Online

Accepted55 / 55 testcases passed
Sahil Khan submitted at Jul 09, 2026 17:16

AnalysisSolution

Runtime
0 ms | Beats 100.00%

Memory
42.83 MB | Beats 83.57%



Code | Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = val; t
9  * }
```

View more

Write your notes here

Code

JavaAuto

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = val; this.next =
9  *     next; }
10 * }
11
12 class Solution {
13     public ListNode swapPairs(ListNode head) {
14         if(head==null || head.next==null) return head;
15
16         ListNode curr = head;
17
18         while(curr!=null && curr.next!=null){
19             int temp = curr.val;
20             curr.val = curr.next.val;
21             curr.next.val = temp;
22
23             curr = curr.next.next;
24         }
25
26         return head;
27     }
28 }
```

SavedLn 1, Col 1

TestcaseTest Result

You must run your code first